Лабораторная работа 3
по дисциплине «ООП»

Выполнила:
Филатова Анастасия Анатольевна,
студентка группы 6301-030301D,
институт информатики и кибернетики;

Самара 2025

Цель лабораторной работы: Дополнить пакет для работы с функциями одной переменной, заданными в табличной форме, добавив классы исключений, новый класс функций и базовый интерфейс.

Задание 1

Классы исключенийI:

java.lang.Exception – базовый класс исключений

java.lang.IndexOutOfBoundsException – исключение выхода за границы индекса java.lang.ArrayIndexOutOfBoundsException – исключение выхода за границы массива java.lang.IllegalArgumentException – исключение недопустимого аргумента java.lang.IllegalStateException – исключение недопустимого состояния объекта

Задание 2

Созданы два класса исключений в пакете functions:

FunctionPointIndexOutOfBoundsException наследует от IndexOutOfBoundsException

```
package functions;


public class FunctionPointIndexOutOfBoundsException extends IndexOutOfBoundsException {
```

IndexOutOfBoundsException InappropriateFunctionPointException наследует от Exception

```
package functions;


public class InappropriateFunctionPointException extends Exception {
```

Реализованы необходимые конструкторы.

Задание 3

Класс TabulatedFunction модифицирован для выбрасывания исключений:

Конструкторы выбрасывают IllegalArgumentException при: Левая граница области определения ≥ правой границы Количество точек < 2

Методы доступа (getPoint(), setPoint(), и др.) выбрасывают FunctionPointIndexOutOfBoundsException при недопустимом индексе.

Методы setPoint() и setPointX() выбрасывают InappropriateFunctionPointException при нарушении упорядоченности точек по X.

Метод deletePoint() выбрасывает IllegalStateException при попытке удаления точки, если останется менее 3 точек.

Задания 4 и 5:

Реализован класс LinkedListTabulatedFunction, использующий двусвязный циклический список с выделенной головой. (это структура, в которой объект элемента списка хранит две ссылки — на предыдущий элемент и на следующий элемент, а голова списка не используется для хранения данных и присутствует в списке всегда (пустой список — это объект головы, ссылающийся сам на себя и как на предыдущий элемент, и как на следующий).)

Внутренний класс FunctionNode (приватный статический класс):

```
private static class FunctionNode {
    FunctionPoint point;    // Данные точки функции
    FunctionNode prev;      // Ссылка на предыдущий узел
    FunctionNode next;      // Ссылка на следующий узел
}
```

Класс инкапсулирован внутри LinkedListTabulatedFunction и недоступен извне,

что обеспечивает защиту целостности структуры данных.

Конструкторы:

1. LinkedListTabulatedFunction(double leftX, double rightX, int pointCount) -
   создает равномерную сетку точек с функцией y = sin(x)
2. LinkedListTabulatedFunction(double[] xValues, double[] yValues) -
   создает функцию из массивов координат

Метод getNodeByIndex(int index)

Данный метод реализует интеллектуальный доступ к элементам списка:

Проверяет корректность индекса ($0 \leq index < size$)

Использует кэширование: хранит ссылку на последний запрошенный узел и его индекс

Выбирает оптимальное направление обхода:

Если запрашиваемый индекс близок к последнему доступному – начинает от него

Если индекс ближе к началу – начинает от головы

Метод addNodeToTail()

Добавляет новый узел в конец списка:

Создаёт новый узел

Настраивает связи: новый узел становится между текущим хвостом и головой

Увеличивает счётчик размера

Возвращает ссылку на созданный узел

Метод addNodeByIndex(int index)

Вставляет узел в произвольную позицию:

Обрабатывает крайние случаи (добавление в начало или конец)

Находит узел, после которого нужно вставить новый элемент

Корректирует связи четырёх узлов: предыдущего, нового и следующего

Обновляет кэш последнего доступа при необходимости

Метод deleteNodeByIndex(int index)

Удаляет узел по указанному индексу:

Проверяет возможность удаления (минимум 3 точки должны оставаться)

Находит удаляемый узел через getNodeByIndex()

Перенастраивает связи: предыдущий узел связывается со следующим

Освобождает ресурсы (Java сборщик мусора удалит узел)

Корректирует кэш последнего доступа

Задание 6

Класс TabulatedFunction переименован в ArrayTabulatedFunction

Создан интерфейс TabulatedFunction, содержащий объявления всех общих методов:

```
public interface TabulatedFunction {
    int getPointCount();
    double getPointX(int index);
    double getPointY(int index);
```

void setPointX(int index, double x);

// ... другие методы}

Оба класса (ArrayTabulatedFunction и LinkedListTabulatedFunction) реализуют созданный интерфейс.

Задание 7

Создан тестовый класс Main для проверки работы обоих классов. Тестирование включает:

Корректное создание функций через оба конструктора

Проверка работы методов доступа и модификации

Тестирование исключений во всех предусмотренных случаях

Вывод:

PS C:\Users\79170\Desktop\Мои проекты\lab1\Lab-3-2025> java -cp out Main

=== Testing with TabulatedFunction interface ===

1. Testing ArrayTabulatedFunction through TabulatedFunction interface:

Created ArrayTabulatedFunction with sin(x) values

Function: {(0.0; 0.0), (1.0; 0.8414709848078965), (2.0; 0.9092974268256817), (3.0; 0.1411200080598672), (4.0; -0.7568024953079282), (5.0; -0.9589242746631385), (6.0; -0.27941549819892586), (7.0; 0.6569865987187891), (8.0; 0.9893582466233818), (9.0; 0.4121184852417566), (10.0; -0.5440211108893698)}

Point count: 11

Basic operations:

Point at index 0: (0.0; 0.0)

Point at index 5: (5.0; -0.9589242746631385)

Last point: (10.0; -0.5440211108893698)

Function values:

f(2.5) = 0.5252087174427744

f(5.0) = -0.9589242746631385

f(15.0) (outside range) = NaN

Modification tests:

After setPointY(3, 10.0): (3.0; 10.0)

Successfully added point (1.5, 0.5)

New point count: 12

Successfully deleted point: (1.5; 0.5)

Point count after deletion: 11

2. Testing LinkedListTabulatedFunction through TabulatedFunction interface:

Created LinkedListTabulatedFunction with sin(x) values

Function: {(0.0; 0.0), (1.0; 0.8414709848078965), (2.0; 0.9092974268256817), (3.0; 0.1411200080598672), (4.0; -0.7568024953079282), (5.0; -0.9589242746631385), (6.0; -0.27941549819892586), (7.0; 0.6569865987187891), (8.0; 0.9893582466233818), (9.0; 0.4121184852417566), (10.0; -0.5440211108893698)}

Point count: 11

Basic operations:

Point at index 0: (0.0; 0.0)

Point at index 5: (5.0; -0.9589242746631385)

Last point: (10.0; -0.5440211108893698)

Function values:

f(2.5) = 0.5252087174427744

f(5.0) = -0.9589242746631385

f(15.0) (outside range) = NaN

Modification tests:

After setPointY(3, 10.0): (3.0; 10.0)

Successfully added point (1.5, 0.5)

New point count: 12

Successfully deleted point: (1.5; 0.5)

Point count after deletion: 11

3. Testing exception cases:

Testing ArrayTabulatedFunction exceptions through interface:

1. IllegalArgumentException caught: Invalid parameters: leftX must be less than rightX and pointCount >= 2

2. IllegalArgumentException caught: Invalid parameters: leftX must be less than rightX and pointCount >= 2

3. FunctionPointIndexOutOfBoundsException caught: Function point index out of bounds: 10

4. InappropriateFunctionPointException caught: Inappropriate function point with x = -1.0

5. IllegalStateException caught: Cannot delete point: minimum 3 points required

6. InappropriateFunctionPointException caught: Inappropriate function point with x = 5.0

Testing LinkedListTabulatedFunction exceptions through interface:

1. IllegalArgumentException caught: Invalid parameters: leftX must be less than rightX and pointCount >= 2

2. IllegalArgumentException caught: Invalid parameters: leftX must be less than rightX and pointCount >= 2

3. FunctionPointIndexOutOfBoundsException caught: Function point index out of bounds: 10

4. InappropriateFunctionPointException caught: Inappropriate function point with x = -1.0

5. IllegalStateException caught: Cannot delete point: minimum 3 points required

6. InappropriateFunctionPointException caught: Inappropriate function point with x = 5.0